



Universidad Nacional Autónoma de México



Facultad de Ingeniería

Integrantes:

García Cortés Adolfo de Jesús - 320317264

Lara Hernandez Angel Husiel - 320060829

Lugo Manzano Rodrigo - 320206968

Sistemas Distribuidos

Grupo: 01 - Semestre: 2026-2

Proyecto final

Simulación de Procesos Distribuidos usando Docker, gRPC y
Relojes vectoriales.

Profesora:

M.I. Elba Karen Saenz García

Fecha de Entrega:

10 de Junio del 2026

Índice

1	Objetivo	1
2	Planteamiento del escenario	1
3	Diagrama del escenario	2
4	Documentacion de código	4
4.1	Código proto	4
4.1.1	Descripción del Servicio	5
4.1.2	Descripción de Mensajes	5
4.2	Códigos python	6
4.2.1	Código 1: <code>vectorial.py</code> — Reloj Vectorial	6
4.2.2	Código 2: <code>bitacora.py</code> — Manejador de Logs	7
4.2.3	Código 3: <code>process.py</code> — Servidor gRPC	8
4.2.4	Código 4: <code>driver.py</code> — Orquestador del Escenario	14
4.2.5	Código 5: <code>cli.py</code> — Cliente Interactivo	15
4.3	Códigos docker	18
5	Secuencia de comandos para la ejecución	21
5.1	Preparación del entorno	21
5.2	Secuencia de comandos para el escenario esto, para la terminal 2	21
5.3	Revision de logs	24
6	Resultados	25
6.1	Capturas de los resultados	25
6.2	Bitacoras	31
7	Conclusión	34
8	Link de video	34
	Referencias	34



1. Objetivo

Comprender cómo se mantiene el orden lógico de eventos en sistemas distribuidos mediante el uso de relojes vectoriales.

2. Planteamiento del escenario

Para plantear el escenario, nos basamos en el funcionamiento de las aplicaciones de reparto de comida utilizando una arquitectura de microservicios. Más precisamente, simulamos el flujo de una orden en el restaurante Carl's Jr., eligiendo como pedido específico la hamburguesa Famous Star. El escenario abarca desde que el usuario selecciona su comida hasta que el sistema manda la notificación final de entrega, asignando a cada proceso una responsabilidad específica.

Todo comienza en el Proceso 1, que representa la Aplicación Cliente y simula la interfaz de usuario. Aquí, el evento interno consiste en agregar la comida al carrito de compras junto con el monto a pagar. Una vez que la orden está armada, el Proceso 1 envía una solicitud de cobro mediante un mensaje punto a punto al Proceso 2, el cual funciona como el movimiento de pago para verificar si la tarjeta es válida y tiene fondos. En este paso se transmite el reloj vectorial inicial para empezar la sincronización temporal del sistema.

Después, al recibir la solicitud, el Proceso 2 actualiza su reloj local y ejecuta el evento interno de validar los datos de la tarjeta y procesar la transacción. Tras aprobar el pago, este proceso se comunica con el Proceso 3, que simula la aplicación en la cocina del restaurante, enviándole la confirmación de los fondos y los detalles de la orden para saber qué tipo de hamburguesa desea el cliente. Esto garantiza un orden lógico, asegurando que el restaurante no comience a cocinar sin que el pago haya sido exitoso.

Luego, el Proceso 3 registra un evento interno simulando la preparación y el empaquetado de la orden. Una vez que el paquete está listo en el mostrador, la cocina envía una notificación directa al Proceso 4, que representa la aplicación del repartidor. Este cuarto proceso recibe la instrucción, sincroniza su vector y lanza su propio evento interno para calcular la ruta óptima de entrega mediante GPS hacia el domicilio del cliente. Al iniciar el viaje, el repartidor actualiza el estado de la orden mandando un mensaje al último proceso.

Ya por último, tenemos al Proceso 5, que es el servidor de notificaciones. Al recibir el aviso del repartidor, ejecuta un evento interno para registrar el tiempo estimado de llegada o ETA. Para terminar el ciclo de la orden, el Proceso 5 emite una difusión masiva hacia los cuatro procesos restantes para indicar que la orden ha sido completada y entregada de forma exitosa.

Este ultimo paso es importante, debido a que es el punto donde el sistema logra sincronizar completamente todos los estados haciendo que los cinco relojes vectoriales sean los mismos de manera simultánea.



Servicios gRPC

1. **Enviar Mensaje:** Recibe una estructura ReqEnvio con los identificadores de origen, destino y de mensaje, esto de forma estructurada como una cadena, después de esto se incrementa la posición en el Reloj Vectorial del emisor y coloca la línea de envío en la bitácora.
2. **Recibir Mensaje:** Recibe una estructura ReqRecepcion que contiene el proceso de origen, el mensaje y el arreglo del vector, después de esto actualiza el reloj local calculando los valores máximos, esto por de forma que lo hace componente a componente, e incrementando en 1, por último registra el evento de recepción y devuelve un mensaje de confirmación con el vector del reloj con el tiempo actual.
3. **Evento Interno:** Recibe una solicitud ReqInterno con la descripción de una tarea (armado de carrito, validación de pago, preparación en cocina o cálculo de ruta GPS), después este incrementa la posición en su reloj lógico vectorial y pone de forma síncrona la entrada en el archivo de registro.
4. **Broadcast:** Recibe una petición de difusión masiva, este mismo incrementa el reloj vectorial del emisor en 1 y atiende de manera por igual al mismo tiempo el estado actual junto con el mensaje a todos los demás procesos para sincronizar el estado de la orden.

3. Diagrama del escenario

Descripción del reloj vectorial

Para entender cómo cambia el reloj vectorial a lo largo del escenario, es importante seguir el flujo de los mensajes paso a paso. Al principio, los cinco procesos inician con sus relojes completamente en ceros.

Todo comienza cuando el Proceso 1 realiza la acción de armar el carrito. Al ser un evento interno, solamente suma uno en la primera posición de su arreglo, quedando en $[1,0,0,0,0]$. Enseguida, el Proceso 1 envía la petición de cobro al Proceso 2. Para mandar el mensaje, el Proceso 1 vuelve a incrementar su contador a $[2,0,0,0,0]$ y adjunta este vector en la transmisión.

Cuando el Proceso 2 recibe el mensaje, compara el vector que le llegó con el suyo que estaba en ceros. Toma los valores más altos y le suma uno a su propia posición, que es la segunda, por lo que su reloj avanza a $[2,1,0,0,0]$. Después, el Proceso 2 hace la validación interna de la tarjeta y simplemente suma otro número a su contador local, pasando a $[2,2,0,0,0]$. Al terminar, manda la confirmación de pago al restaurante en el Proceso 3, lo que hace que el reloj del Proceso 2 suba a $[2,3,0,0,0]$ para el envío.

El Proceso 3 recibe los datos del pago y sincroniza su reloj tomando los valores máximos, sumando uno en la tercera posición para quedar en $[2,3,1,0,0]$. Luego, el restaurante prepara la comida mediante un evento interno y su reloj avanza a $[2,3,2,0,0]$. Cuando la comida está lista, el Proceso 3 manda el aviso al repartidor en el Proceso 4, subiendo su propio contador a $[2,3,3,0,0]$ al enviar el paquete de datos.

El repartidor, que es el Proceso 4, recibe la notificación del restaurante, hace la comparación de máximos y aumenta su cuarta posición, actualizando su reloj a $[2,3,3,1,0]$. A continuación, ejecuta su evento interno para calcular la ruta, lo que sube su contador a $[2,3,3,2,0]$. Al iniciar el viaje hacia el cliente, le manda el aviso al servidor central en el Proceso 5, por lo que su reloj avanza a $[2,3,3,3,0]$ en el momento del envío.

El servidor recibe el aviso del viaje y actualiza su reloj a $[2,3,3,3,1]$ sumando uno en la quinta posición. Después realiza el evento interno para calcular el tiempo estimado de llegada, avanzando a $[2,3,3,3,2]$.

Finalmente, ocurre la difusión masiva. El servidor prepara el mensaje final para todos, subiendo su contador a $[2,3,3,3,3]$ y enviando este vector a los otros cuatro procesos al mismo tiempo. Cuando el Proceso 1 recibe esta difusión, compara el vector nuevo con el último que tenía guardado, toma todos los máximos y le suma uno a su primera posición, terminando en $[3,3,3,3,3]$. Los demás procesos hacen exactamente lo mismo, sumando uno a su respectiva posición, lo que nos demuestra cómo todos los nodos logran enterarse de la historia completa y mantener el orden causal de nuestro sistema distribuido.

Diagrama

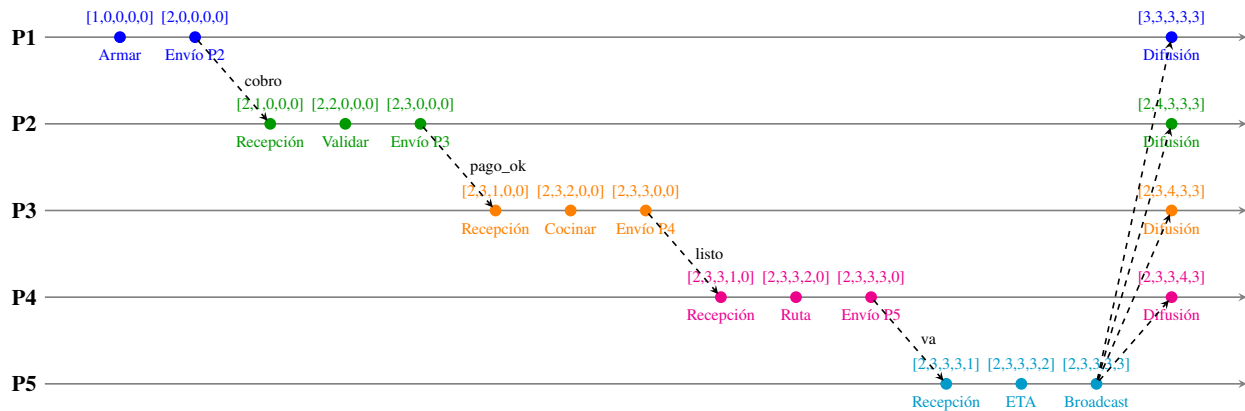


Figura 1: Diagrama de secuencias y actualización de relojes vectoriales del escenario propuesto.

4. Documentacion de código

4.1. Código proto

Define cómo se comunican las diferentes partes del sistema utilizando **Protocol Buffers v3**. Para ello, se especifica el servicio **ProcesoService**, que incluye cuatro operaciones remotas (RPC) y los mensajes de entrada y salida que utiliza cada una.

```
1  syntax = "proto3";
2  package procesos;
3
4  service ProcesoService {
5      rpc EnviarMensaje (ReqEnvio) returns (Ack);
6      rpc RecibirMensaje (ReqRecepcion) returns (Ack);
7      rpc EventoInterno (ReqInterno) returns (Ack);
8      rpc Broadcast (ReqDifusion) returns (Ack);
9  }
10
11 message ReqEnvio {
12     string origen = 1;
13     string destino = 2;
14     string msg = 3;
15 }
16
17 message ReqRecepcion {
18     string origen = 1;
19     string msg = 2;
20     repeated int64 vector = 3;
21 }
22
23 message ReqInterno {
24     string desc = 1;
25 }
26
27 message ReqDifusion {
28     string origen = 1;
29     string msg = 2;
30 }
31
32 message Ack {
33     bool ok = 1;
34     string msg = 2;
35     repeated int64 vector = 3;
36 }
```

Listing 1: procesos.proto - Definición IDL del servicio gRPC



4.1.1. Descripción del Servicio

RPC	Entrada	Salida	Descripción
EnviarMensaje	ReqEnvio	Ack	Solicita al nodo que envíe un mensaje a otro.
RecibirMensaje	ReqRecepcion	Ack	Notifica al nodo la llegada de un mensaje.
EventoInterno	ReqInterno	Ack	Dispara un evento local de negocio.
Broadcast	ReqDifusion	Ack	Difunde un mensaje a todos los nodos.

4.1.2. Descripción de Mensajes

Mensaje	Campo	Tag	Descripción
ReqEnvio	origen	1	ID del proceso emisor.
	destino	2	ID del proceso receptor.
	msg	3	Contenido del mensaje (JSON).
ReqRecepcion	origen	1	ID del proceso que origina el mensaje.
	msg	2	Contenido del mensaje.
	vector	3	Reloj vectorial del emisor (repeated int64).
ReqInterno	desc	1	Descripción del evento interno.
ReqDifusion	origen	1	ID del proceso que inicia la difusión.
	msg	2	Contenido a difundir.
Ack	ok	1	true si la operación tuvo éxito.
	msg	2	Mensaje descriptivo del resultado.
	vector	3	Estado actual del reloj vectorial del nodo (repeated int64).

El campo vector en ReqRecepcion y Ack utiliza repeated int64 para enviar los valores del reloj vectorial como un arreglo. En Python, este campo se representa automáticamente como una lista de números enteros (list[int]).



4.2. Códigos python

4.2.1. Código 1 : vectorial.py — Reloj Vectorial

Implementa la clase `RelojVectorial`, que se encarga de administrar el reloj vectorial de un nodo dentro de un sistema distribuido. Este reloj consiste en un arreglo de enteros, donde cada posición almacena el contador lógico correspondiente a uno de los procesos del sistema.

Dado un sistema con n procesos, el reloj vectorial de un proceso P_i se representa mediante un vector V de n enteros, donde cada posición almacena el contador lógico de un proceso. Su funcionamiento se basa en tres reglas principales:

- **Evento local:** el proceso incrementa su propio contador.
- **Envío de mensaje:** antes de enviar un mensaje, el proceso incrementa su contador y adjunta el reloj vectorial actual al mensaje.
- **Recepción de mensaje:** al recibir un mensaje, el proceso incrementa su contador y actualiza su reloj comparando cada posición con la recibida, conservando siempre el valor más grande.

```
1 import threading
2
3 class RelojVectorial:
4     def __init__(self, id_proc, num_proc=5):
5         self.num_proc = num_proc
6         self.idx = int(id_proc.replace("P", "")) - 1
7         self.vec = [0] * num_proc
8         self._bloqueo = threading.Lock()
9
10    def evento(self):
11        with self._bloqueo:
12            self.vec[self.idx] += 1
13            return list(self.vec)
14
15    def actualizar(self, vec_recibido):
16        with self._bloqueo:
17            self.vec[self.idx] += 1
18            for i in range(self.num_proc):
19                self.vec[i] = max(self.vec[i], vec_recibido[i])
20            return list(self.vec)
21
22    def valor(self):
23        with self._bloqueo:
24            return list(self.vec)
```

Listing 2: vectorial.py — Implementación del Reloj Vectorial



`__init__(self, id_proc, num_proc=5)`

Constructor de la clase. Inicializa el reloj vectorial del proceso, crea el vector con todos sus valores en cero y configura los elementos necesarios para acceder al reloj de forma segura desde varios hilos.

`evento(self) -> list`

Registra un evento local en el proceso. Para ello, incrementa el contador correspondiente al proceso actual y devuelve una copia del reloj vectorial actualizado. La operación está protegida mediante un Lock para evitar accesos simultáneos.

`actualizar(self, vec_recibido) -> list`

Procesa la recepción de un mensaje que viene de otro nodo. Incrementa el contador local y actualiza el reloj vectorial comparando cada posición con el vector recibido, conservando siempre el valor más alto. Finalmente, devuelve una copia del vector actualizado. La operación está protegida mediante un Lock.

`valor(self) -> list`

Devuelve una copia del estado actual del reloj vectorial sin realizar ninguna modificación. Este método se utiliza para consultar el reloj de forma segura y está protegido mediante un Lock.

4.2.2. Código 2: bitacora.py — Manejador de Logs

La clase Bitacora se encarga de almacenar mensajes en un archivo de registro (log) de forma segura. Para evitar problemas cuando varios hilos intentan escribir al mismo tiempo, utiliza un `threading.Lock`, garantizando que cada mensaje se guarde correctamente sin alterar el contenido del archivo.

```
1  import os
2  import threading
3  class Bitacora:
4      def __init__(self, ruta):
5          self.ruta = ruta
6          self._bloqueo = threading.Lock()
7          dir_ruta = os.path.dirname(self.ruta)
8          if dir_ruta:
9              os.makedirs(dir_ruta, exist_ok=True)
10
11     def agregar(self, linea):
12         linea_segura = linea.rstrip("\n")
13         with self._bloqueo:
14             with open(self.ruta, "a", encoding="utf-8") as arch:
15                 arch.write(linea_segura + "\n")
```

Listing 3: bitacora.py — Escritura concurrente de logs



`__init__(self, ruta)`

Constructor de la bitácora. Recibe la ubicación del archivo de registro y, si los directorios necesarios no existen, los crea automáticamente para garantizar que el archivo pueda almacenarse correctamente.

Parámetro	Tipo	Descripción
<code>ruta</code>	<code>str</code>	Ruta donde se almacenará el archivo de log.

`agregar(self, línea)`

Agrega una nueva línea al archivo de log de forma segura. Antes de escribir, elimina los saltos de línea sobrantes para evitar espacios en blanco innecesarios. El archivo se abre en modo **append** ({“a”}), de manera que los nuevos registros se añaden al final sin modificar el contenido existente.

4.2.3. Código 3: `process.py` — Servidor gRPC

Es el componente principal del sistema. Implementa el servicio gRPC `ServicioProceso` y se encarga de coordinar la comunicación entre los nodos. Sus funciones incluyen el envío y recepción de mensajes, la ejecución de eventos internos, la difusión de mensajes a todos los nodos (*broadcast*) y la actualización del reloj vectorial cada vez que ocurre alguna de estas operaciones.

Funciones Auxiliares

```
1 def leer_nodos(crudos):
2     nodos = {}
3     if not crudos: return nodos
4     items = [i.strip() for i in crudos.split(",") if i.strip()]
5     for i in items:
6         if "=" not in i: continue
7         id_nodo, dir_nodo = i.split("=", 1)
8         nodos[id_nodo.strip()] = dir_nodo.strip()
9     return nodos
10
11 def obt_env(clave, defecto=None):
12     val = os.getenv(clave)
13     return val if val not in (None, "") else defecto
14
15 def ruta_log(id_proc):
16     return obt_env("LOG_PATH", f"/app/logs/{id_proc}.log")
```

Listing 4: `process.py` — Funciones auxiliares



Función	Descripción
<code>leer_nodos(crudos)</code>	Convierte una cadena con el formato ID=host:puerto en un diccionario que relaciona cada proceso con el host y puerto que utiliza para comunicarse.
<code>obt_env(clave, defecto)</code>	Obtiene el valor de una variable de entorno. Si la variable no existe o está vacía, devuelve el valor indicado por defecto.
<code>ruta_log(id_proc)</code>	Genera la ruta del archivo de log asociado al proceso. Si la variable de entorno LOG_PATH está definida, se utiliza como directorio base para almacenar el archivo.

Clase ServicioProceso

```
1 COLORES = {
2     "P1": "\033[94m", "P2": "\033[92m", "P3": "\033[93m",
3     "P4": "\033[95m", "P5": "\033[96m",
4 }
5 FIN_COL = "\033[0m"
6 class ServicioProceso(procesos_pb2_grpc.ProcesoServiceServicer):
7     def __init__(self, id_proc, nodos, bita):
8         self.id_proc = id_proc
9         self.nodos = nodos
10        self.bita = bita
11        self.reloj = RelojVectorial(id_proc, len(nodos))
12        self.color = COLORES.get(id_proc, "")
13
14    def _log(self, linea):
15        self.bita.agregar(linea)
16        print(f"{self.color}{linea}{FIN_COL}", flush=True)
17
18    def _tarea(self, desc):
19        txt = desc.lower()
20        if "carrito" in txt or "armar" in txt:
21            return '{"accion":"crear_carrito", "item":"Super Star", "precio":250}'
22        if "pago" in txt or "cobro" in txt or "tarjeta" in txt:
23            return '{"accion":"validar_tarjeta", "status":"ok", "auth":"TXN-9981"}'
24        if "cocina" in txt or "preparar" in txt:
25            return '{"accion":"cocina", "status":"empaquetado", "temp":"caliente"}'
26        if "ruta" in txt or "gps" in txt:
27            return '{"accion":"gps", "destino":"Tlalnepantla", "dist":"4.2km"}'
28        if "tiempo" in txt or "notificacion" in txt:
29            return '{"accion":"push", "disp":"Celular", "status":"enviado'}'
30        return f'{{"len_desc":{len(desc)}}}'
```

Listing 5: process.py — Constructor y métodos privados



Constructor `__init__`

Inicializa el servicio del proceso, configurando su identificador, la información de los demás nodos del sistema, la bitácora y las estructuras necesarias para gestionar la comunicación y el reloj vectorial.

Parámetro	Tipo	Descripción
id_proc	str	Identificador del proceso (por ejemplo, "P1").
nodos	dict	Diccionario con la información de conexión de todos los procesos.
bita	Bitacora	Instancia utilizada para registrar eventos en el archivo de log.

Método privado `_log(linea)`

Registra una línea de texto tanto en la bitácora como en la consola. Además, utiliza colores para diferenciar los mensajes de cada proceso y fuerza la impresión inmediata mediante `flush=True`.

Método privado `_tarea(desc)`

Simula el trabajo realizado por el nodo a partir de la descripción de un evento. Dependiendo del contenido de la descripción, ejecuta una acción específica y devuelve el resultado en formato JSON.

Métodos gRPC del Servicer

```
1  def _enviar(self, destino, msg):
2      if destino not in self.nodos:
3          return False, f"Desconocido: {destino}", self.reloj.valor()
4      if destino == self.id_proc:
5          return False, "Auto-envio", self.reloj.valor()
6
7      vec = self.reloj.evento()
8      str_v = str(vec).replace(" ", "")
9      self._log(f'[SEND] {self.id_proc} -> {destino} msg="{msg}" vector
      r={str_v}')
10
11     dir_nodo = self.nodos[destino]
12     req = procesos_pb2.ReqRecepcion(origen=self.id_proc, msg=msg, vector=vec)
13     try:
14         with grpc.insecure_channel(dir_nodo) as canal:
15             stub = procesos_pb2_grpc.ProcesoServiceStub(canal)
16             res = stub.RecibirMensaje(req)
17             str_ack = str(list(res.vector)).replace(" ", "")
18             self._log(f'[ACK] {self.id_proc} <- {destino} '
```



```

19         f'confirmacion="{res.msg}" vector_ack={str_ack}'))
20         return True, res.msg, list(res.vector)
21     except grpc.RpcError as err:
22         return False, err.details() or "Error gRPC", self.reloj.valor()
23
24     def EnviarMensaje(self, req, ctx):
25         if req.origen and req.origen != self.id_proc:
26             ctx.set_code(grpc.StatusCode.INVALID_ARGUMENT)
27             return procesos_pb2.Ack(ok=False, msg="origen invalido",
28                                     vector=self.reloj.valor())
29         ok, msg, _ = self._enviar(req.destino, req.msg)
30         if not ok: ctx.set_code(grpc.StatusCode.FAILED_PRECONDITION)
31         return procesos_pb2.Ack(ok=ok, msg=msg, vector=self.reloj.valor())
32
33     def RecibirMensaje(self, req, ctx):
34         vec_rec = list(req.vector)
35         str_rec = str(vec_rec).replace(" ", "")
36         vec_act = self.reloj.actualizar(vec_rec)
37         str_act = str(vec_act).replace(" ", "")
38         self._log(f'[RECEIVE] {self.id_proc} <- {req.origen} msg="{req.msg}" '
39                 f'vector_recibido={str_rec} vector_actualizado={str_act}')
40         return procesos_pb2.Ack(ok=True, msg="ACK", vector=vec_act)

```

Listing 6: process.py — Envío y recepción de mensajes

Método	Descripción
<code>_enviar(destino, msg)</code>	Método interno encargado de enviar un mensaje a otro nodo mediante gRPC. Antes del envío actualiza el reloj vectorial, realiza la comunicación y registra la respuesta recibida. También maneja posibles errores durante la comunicación.
<code>EnviarMensaje(req, ctx)</code>	Método gRPC que recibe una solicitud de envío de mensaje. Verifica la información recibida, utiliza el mecanismo de envío correspondiente y devuelve una confirmación de la operación.
<code>RecibirMensaje(req, ctx)</code>	Método gRPC ejecutado cuando un nodo recibe un mensaje. Actualiza el reloj vectorial con la información recibida, registra el evento y devuelve una confirmación junto con el estado actualizado del reloj.

Evento interno y Broadcast

```

1     def EventoInterno(self, req, ctx):
2         desc = req.desc or "evento"

```



```
3     res = self._tarea(desc)
4     vec = self.reloj.evento()
5     str_v = str(vec).replace(" ", "")
6     self._log(f'[INTERNAL] {self.id_proc} vector={str_v}')
7     return procesos_pb2.Ack(ok=True, msg=f"Registro: {res}", vector=vec)
8
9     def Broadcast(self, req, ctx):
10         if req.origen and req.origen != self.id_proc:
11             ctx.set_code(grpc.StatusCode.INVALID_ARGUMENT)
12             return procesos_pb2.Ack(ok=False, msg="origen invalido",
13                                     vector=self.reloj.valor())
14
15         msgs = []
16         vec = self.reloj.evento()
17         str_v = str(vec).replace(" ", "")
18
19         for dest in sorted(self.nodos.keys()):
20             if dest == self.id_proc: continue
21             self._log(f'[SEND] {self.id_proc} -> {dest} msg="{req.msg}" '
22                     f'vector r={str_v}')
23             dir_nodo = self.nodos[dest]
24             peticion = procesos_pb2.ReqRecepcion(origen=self.id_proc,
25                                                  msg=req.msg, vector=vec)
26
27             try:
28                 with grpc.insecure_channel(dir_nodo) as canal:
29                     stub = procesos_pb2_grpc.ProcesoServiceStub(canal)
30                     stub.RecibirMensaje(peticion)
31                     msgs.append(f"{dest}:OK")
32             except Exception:
33                 msgs.append(f"{dest}:ERR")
34
35         resumen = ", ".join(msgs) if msgs else "Sin destinos"
36         return procesos_pb2.Ack(ok=True, msg=resumen, vector=self.reloj.valor())
```

Listing 7: process.py — Evento interno y Broadcast



Método	Descripción
EventoInterno(req, ctx)	Simula una actividad realizada únicamente por el nodo local. Ejecuta la tarea asociada a la descripción recibida, actualiza el reloj vectorial y registra el evento en la bitácora. Devuelve el resultado de la tarea en la respuesta.
Broadcast(req, ctx)	Envía el mismo mensaje a todos los demás nodos del sistema. Antes de iniciar los envíos, obtiene un único valor del reloj vectorial para que todos los destinatarios reciban el mensaje con la misma marca de tiempo lógica.

Función de Arranque iniciar()

```
1 def iniciar():
2     id_proc = obt_env("PROCESS_ID")
3     if not id_proc: raise SystemExit("Falta PROCESS_ID")
4
5     nodos = leer_nodos(obt_env("PEERS", ""))
6     if not nodos: raise SystemExit("Falta PEERS")
7
8     pto = int(obt_env("PORT", puerto))
9     bita = Bitacora(ruta_log(id_proc))
10
11     servidor = grpc.server(futures.ThreadPoolExecutor(max_workers=10))
12     procesos_pb2_grpc.add_ProcesoServiceServicer_to_server(
13         ServicioProceso(id_proc, nodos, bita), servidor)
14     servidor.add_insecure_port(f"0.0.0.0:{pto}")
15     servidor.start()
16
17     col = COLORES.get(id_proc, "")
18     print(f"{col}Nodo {id_proc} activo en el puerto {pto}...{FIN_COL}", flush=True)
19
20     try:
21         while True: time.sleep(3600)
22     except KeyboardInterrupt:
23         servidor.stop(0)
```

Listing 8: process.py — Arranque del servidor gRPC

Lee las variables de entorno PROCESS_ID, PEERS y PORT para obtener la configuración del nodo. Después, crea y pone en marcha el servidor gRPC utilizando un ThreadPoolExecutor con 10 hilos de trabajo. El proceso permanece en ejecución mediante un bucle con pausas periódicas (sleep) y, si se recibe una señal de interrupción (KeyboardInterrupt), el servidor se detiene de forma ordenada antes de finalizar.



4.2.4. Código 4: driver.py — Orquestador del Escenario

Script independiente que ejecuta automáticamente la simulación completa del proceso de entrega de comida. Para ello, realiza las llamadas a los RPCs de los distintos nodos en el orden establecido, incorporando pausas de un segundo entre cada evento para reproducir la secuencia de interacciones del sistema.

```
1  import argparse, os, time, grpc
2  import procesos_pb2, procesos_pb2_grpc
3
4  def leer_nodos(crudos):
5      # (misma logica que en process.py)
6      nodos = {}
7      if not crudos: return nodos
8      items = [i.strip() for i in crudos.split(",") if i.strip()]
9      for i in items:
10         if "=" not in i: continue
11         id_nodo, dir_nodo = i.split("=", 1)
12         nodos[id_nodo.strip()] = dir_nodo.strip()
13     return nodos
14
15 def evt_interno(nodos, dest, desc):
16     dir_nodo = nodos[dest]
17     with grpc.insecure_channel(dir_nodo) as canal:
18         stub = procesos_pb2_grpc.ProcesoServiceStub(canal)
19         stub.EventoInterno(procesos_pb2.ReqInterno(desc=desc))
20
21 def evt_enviar(nodos, origen, dest, msg):
22     dir_nodo = nodos[origen]
23     with grpc.insecure_channel(dir_nodo) as canal:
24         stub = procesos_pb2_grpc.ProcesoServiceStub(canal)
25         stub.Enviamensaje(procesos_pb2.ReqEnvio(
26             origen=origen, destino=dest, msg=msg))
27
28 def evt_difusion(nodos, origen, msg):
29     dir_nodo = nodos[origen]
30     with grpc.insecure_channel(dir_nodo) as canal:
31         stub = procesos_pb2_grpc.ProcesoServiceStub(canal)
32         stub.Broadcast(procesos_pb2.ReqDifusion(origen=origen, msg=msg))
33
34 def escenario(nodos):
35     print("Iniciando Escenario de Entrega...", flush=True)
36     evt_interno(nodos, "P1", "armar carrito pedido");           time.sleep(1)
37     evt_enviar(nodos, "P1", "P2", '{"id":"ORD-77X","total":250,"req":"cobro"}')
38     time.sleep(1)
39     evt_interno(nodos, "P2", "validar cobro tarjeta");           time.sleep(1)
40     evt_enviar(nodos, "P2", "P3", '{"id":"ORD-77X","pago":"ok","item":"Super
```




```
        Star"}}')
41     time.sleep(1)
42     evt_interno(nodos, "P3", "preparar comida");           time.sleep(1)
43     evt_enviar(nodos, "P3", "P4", '{"id":"ORD-77X","paquete":"listo"}')
44     time.sleep(1)
45     evt_interno(nodos, "P4", "calcular ruta gps");           time.sleep(1)
46     evt_enviar(nodos, "P4", "P5", '{"id":"ORD-77X","repartidor":"va","eta":"15m"}')
47     time.sleep(1)
48     evt_interno(nodos, "P5", "enviar notificacion");           time.sleep(1)
49     evt_difusion(nodos, "P5", '{"id":"ORD-77X","fin":"entregado","eta":"15m"}')
50     time.sleep(1)
```

Listing 9: driver.py — Orquestador del escenario de entrega

Función	Descripción
evt_interno(nodos, dest, desc)	Ejecuta un evento interno en el nodo indicado por dest, utilizando la descripción proporcionada.
evt_enviar(nodos, origen, dest, msg)	Solicita al nodo origen el envío de un mensaje hacia el nodo dest.
evt_difusion(nodos, origen, msg)	Solicita al nodo origen que envíe el mismo mensaje a todos los demás nodos del sistema.
escenario(nodos)	Ejecuta de forma automática la secuencia completa de eventos del escenario de entrega, siguiendo el orden establecido e incorporando una pausa de un segundo entre acciones para permitir el procesamiento de los nodos.

4.2.5. Código 5: cli.py — Cliente Interactivo

Herramienta de línea de comandos que permite interactuar de forma manual con cualquier nodo del sistema. Su uso evita la necesidad de modificar el código para realizar pruebas o ejecutar operaciones específicas, ya que emplea argparse para gestionar distintos subcomandos y opciones de ejecución.

```
1  import argparse, os, grpc
2  import procesos_pb2, procesos_pb2_grpc
3
4  def leer_nodos(crudos):
5      nodos = {}
6      if not crudos: return nodos
7      items = [i.strip() for i in crudos.split(",") if i.strip()]
8      for i in items:
9          if "=" not in i: continue
```



```
10     id_nodo, dir_nodo = i.split("=", 1)
11     nodos[id_nodo.strip()] = dir_nodo.strip()
12     return nodos
13
14 def res_destino(dest, nodos):
15     if not dest: return None
16     if dest in nodos: return nodos[dest]
17     if ":" in dest: return dest
18     return None
19
20 def obt_origen(args, nodos):
21     if args.origen: return args.origen
22     if args.destino in nodos: return args.destino
23     return os.getenv("PROCESS_ID", "")
24
25 def ejecutar():
26     p = argparse.ArgumentParser(description="Cliente CLI")
27     p.add_argument("--destino", required=True)
28     p.add_argument("--nodos", default=os.getenv("PEERS", ""))
29     p.add_argument("--origen", default="")
30
31     subs = p.add_subparsers(dest="cmd", required=True)
32     cmd_int = subs.add_parser("interno")
33     cmd_int.add_argument("--desc", default="evento")
34
35     cmd_env = subs.add_parser("enviar")
36     cmd_env.add_argument("--receptor", required=True)
37     cmd_env.add_argument("--msg", required=True)
38
39     cmd_dif = subs.add_parser("difusion")
40     cmd_dif.add_argument("--msg", required=True)
41
42     args = p.parse_args()
43     nodos = leer_nodos(args.nodos)
44     dir_dest= res_destino(args.destino, nodos)
45     if not dir_dest: raise SystemExit("Destino invalido")
46
47     id_ori = obt_origen(args, nodos)
48     if args.cmd in {"enviar", "difusion"} and not id_ori:
49         raise SystemExit("Falta origen")
50
51     with grpc.insecure_channel(dir_dest) as canal:
52         stub = procesos_pb2_grpc.ProcesoServiceStub(canal)
53         if args.cmd == "interno":
54             res = stub.EventoInterno(procesos_pb2.ReqInterno(desc=args.desc))
```



```
55     elif args.cmd == "enviar":
56         res = stub.EnviaMensaje(procesos_pb2.ReqEnvio(
57             origen=id_ori, destino=args.receptor, msg=args.msg))
58     else:
59         res = stub.Broadcast(procesos_pb2.ReqDifusion(
60             origen=id_ori, msg=args.msg))
61
62     st = "OK" if res.ok else "ERR"
63     str_v = str(list(res.vector)).replace(" ", "")
64     print(f"{st}: {res.msg} (vector={str_v})")
```

Listing 10: cli.py — Cliente de línea de comandos

Para mantener una estructura modular y resolver las conexiones de forma dinámica, el cliente divide su lógica en las siguientes funciones:

- **leer_nodos:** Procesa una cadena de texto en crudo (típicamente obtenida de la variable de entorno PEERS) y la transforma en un diccionario que asocia el identificador de cada nodo con su dirección física.
- **res_destino:** Resuelve la dirección a la cual se conectará el cliente. Es flexible, ya que permite buscar el destino en el diccionario de nodos conocidos o aceptar directamente una dirección en formato IP:puerto.
- **obt_origen:** Deduce qué identificador debe utilizarse como remitente del mensaje. Si no se indica explícitamente en los argumentos, intenta inferirlo basándose en el destino o, como último recurso, lee la variable de entorno PROCESS_ID.
- **ejecutar:** Orquesta el flujo principal de la herramienta. Valida que los parámetros obligatorios existan y establece un canal de comunicación (`grpc.insecure_channel`) con el destino. Una vez que el nodo remoto procesa la solicitud (evento interno, envío o difusión), esta función captura la respuesta y muestra en consola el estado de la operación junto con el estado del reloj vectorial resultante.

Subcomando	Flags	Descripción
interno	-desc	Ejecuta un evento interno en el nodo seleccionado utilizando la descripción proporcionada.
enviar	-receptor, -msg	Solicita al nodo seleccionado que envíe un mensaje al receptor especificado.
difusion	-msg	Solicita al nodo seleccionado que envíe el mismo mensaje a todos los demás nodos del sistema.



Ejemplo de uso

```
python cli.py --destino P1 --nodos "P1=localhost:50051,P2=localhost:50052" \  
    enviar --receptor P2 --msg '{"prueba": true}'
```

4.3. Códigos docker

Dockerfile:

El archivo Dockerfile lo que hace es que define las instrucciones de como se va a ir contruyendo la imagen base de los contenedores.

Para esto la primer alinea, lo que ahce es que inicia una imagen optimizada de Python python:3.11-slim, depsues de esto, establece /app como el directorio de trabajo principal dentro del entorno virtual.

Despues de esto, transfiere el archivo de requerimientos e instala las dependencias mediante pip install, utilizando la bandera --no-cache-dir, esto se hace mas que nada para evitar almacenar archivos temporales y reduce el peso del contenedor final, despues de esto, copia los directorios de código que en neustro los tenemos por carpetas separadas, que son proto y src, y configura la variable de entorno PYTHONPATH esto se hace para que el intérprete reconozca los módulos importados.

No obstante en la 13 linea, tenemos lo mas importante, que es directaemtne el lanzamiento de el servidor gRPC, esto por medio de la compilacion inicial del archivo proto, este lo que hara es que procesa el archivo procesos.proto para generar las clases cliente y servidor en Python.

Finalmente, declaramos el uso del puerto 50051 para permitir la comunicacion de el gRPC y definemos como comando por defecto la ejecución del servidor principal src/process.py.

Listing 11: Código dockerfile

```
1 FROM python:3.11-slim  
2  
3 WORKDIR /app  
4  
5 COPY requirements.txt ./  
6 RUN pip install --no-cache-dir -r requirements.txt  
7  
8 COPY proto ./proto  
9 COPY src ./src  
10  
11 ENV PYTHONPATH=/app/src  
12  
13 RUN python -m grpc_tools.protoc -I ./proto \  
14     --python_out=./src \  
15     --grpc_python_out=./src \  
16     ./proto/procesos.proto  
17
```



```
18 EXPOSE 50051
19
20 CMD ["python", "src/process.py"]
```

Docker Compose:

El archivo docker-compose es el que se encarga de organizar directamente toda la estructura completa del sistema, definiendo desde la red aislada y un total de seis servicios, donde en este caso son cinco procesos de procesamiento p1 a p5 y un proceso o nodo que es el cliente.

Para cada uno de los procesos servidores, el código lo que hace, es que indica que la imagen debe construirse a partir del directorio actual y le asigna un nombre de contenedor, esto por medio del comando environment, después de esto se colocan ya directamente las variables de entorno para la lógica del código en Python, que en este caso son las de los requisitos del proyecto, tales como el identificador único del proceso `PROCESS_ID`, el mapa de la red indicando las rutas de todos los nodos y la ruta de destino para almacenar directamente las bitácoras.

Para asegurar que los archivos generados no se pierdan al apagar los contenedores, se utilizan volúmenes directamente, que lo que hace es que, mapea un directorio local hacia dentro del propio contenedor.

No obstante, el servicio cliente es casi igual, pero sobrescribe el comando de inicio por defecto con `"sleep.e''infinity"`, esto es por que lo que queremos es que mantenga en un estado de reposo para poder seguir mintiendo comandos de prueba, que son los **docker exec**.

Finalmente, en la parte inferior del archivo, se define la red_privada bajo un formato de tipo puente con una subred que en este caso es parecido al usado directamente en clase en los ejercicios, siendo para este proyecto el 172.21.0.0/16, lo cual permite usar direcciones estáticas para cada contenedor.

Listing 12: Código docker compose

```
1 services:
2   p1:
3     build: .
4     container_name: p1
5     environment:
6       PROCESS_ID: P1
7       PEERS: P1=p1:50051,P2=p2:50051,P3=p3:50051,P4=p4:50051,P5=p5:50051
8       LOG_PATH: /app/logs/p1.log
9     volumes:
10      - ./logs:/app/logs
11     networks:
12       red_privada:
13         ipv4_address: 172.21.0.2
14
15   p2:
16     build: .
```



```
17 container_name: p2
18 environment:
19     PROCESS_ID: P2
20     PEERS: P1=p1:50051,P2=p2:50051,P3=p3:50051,P4=p4:50051,P5=p5:50051
21     LOG_PATH: /app/logs/p2.log
22 volumes:
23     - ./logs:/app/logs
24 networks:
25     red_privada:
26         ipv4_address: 172.21.0.3
27
28 p3:
29     build: .
30     container_name: p3
31     environment:
32         PROCESS_ID: P3
33         PEERS: P1=p1:50051,P2=p2:50051,P3=p3:50051,P4=p4:50051,P5=p5:50051
34         LOG_PATH: /app/logs/p3.log
35     volumes:
36         - ./logs:/app/logs
37     networks:
38         red_privada:
39             ipv4_address: 172.21.0.4
40
41 p4:
42     build: .
43     container_name: p4
44     environment:
45         PROCESS_ID: P4
46         PEERS: P1=p1:50051,P2=p2:50051,P3=p3:50051,P4=p4:50051,P5=p5:50051
47         LOG_PATH: /app/logs/p4.log
48     volumes:
49         - ./logs:/app/logs
50     networks:
51         red_privada:
52             ipv4_address: 172.21.0.5
53
54 p5:
55     build: .
56     container_name: p5
57     environment:
58         PROCESS_ID: P5
59         PEERS: P1=p1:50051,P2=p2:50051,P3=p3:50051,P4=p4:50051,P5=p5:50051
60         LOG_PATH: /app/logs/p5.log
61     volumes:
62         - ./logs:/app/logs
63     networks:
64         red_privada:
65             ipv4_address: 172.21.0.6
66
67 cliente:
68     build: .
```



```
69 container_name: cliente
70 command: ["sleep", "infinity"]
71 environment:
72     PEERS: P1=p1:50051,P2=p2:50051,P3=p3:50051,P4=p4:50051,P5=p5:50051
73 volumes:
74     - ./logs:/app/logs
75 networks:
76     red_privada:
77         ipv4_address: 172.21.0.10
78
79 networks:
80     red_privada:
81         driver: bridge
82         ipam:
83             config:
84                 - subnet: 172.21.0.0/16
```

5. Secuencia de comandos para la ejecución

5.1. Preparación del entorno

Levantar los contenedores: Se levanta todo el sistema la mediante, el siguiente comando, donde la bandera -d libera la terminal.

```
docker compose up -d -build
```

Ver los logs en vivo (Terminal 1):

En dado caso, de que si quiera ver los logs, en vivo se hara uso de dos terminal, no obstante como bien sabemos los logs se guardan en una carpeta llamada logs, y se puede ver con el uso del comando cat, regresando a la terminal 1, se dejara con este comando activo. Esto permite observar cómo los contenedores reaccionan en tiempo real.

```
docker compose logs -f
```

5.2. Secuencia de comandos para el escenario esto, para la terminal 2

1. P1 (App Cliente) – Evento interno: crear el carrito

Inicia el evento interno en P1 simulando que la app del cliente crea la orden.

P1 incrementa su propia posición en el reloj vectorial, pasando de $[0,0,0,0,0]$ a $[1,0,0,0,0]$.

en este punto no hay comunicación con otros procesos.



Comando:

```
docker exec -it cliente python src/cli.py --destino P1 interno --desc "armar carrito pedido"
```

2. P1 → P2 - solicitud de cobro

P1 envía un mensaje gRPC punto a punto a P2 con los datos de la orden.

Al enviarlo, P1 incrementa su reloj nuevamente y envía su vector al mensaje.

Al recibirlo, P2 compara los vectores numero a numero del arreglo, toma los máximos e incrementa su propia posición.

Comando:

```
docker exec -it cliente python src/cli.py --destino P1 enviar --receptor P2 --msg '{"id":"ORD-77X",  
"total":250, "req":"cobro"}
```

3. P2 (Verficiacion de Pago)

P2 ejecuta un evento interno simulando que se verifica si tarjeta de crédito tiene fondos y es real.

Solo incrementa su posición en el reloj vectorial.

Comando:

```
docker exec -it cliente python src/cli.py --destino P2 interno --desc "validar cobro tarjeta"
```

4. P2 → P3 - Confirmacion al restaurante

Con el pago aprobado, P2 le envía los detalles de la orden a P3 vía gRPC.

P3 recibe el vector de P2, aplica lo mismo de revisar los máximos e incrementa su propia posición.

Su reloj ahora esta sincornizado con los eventos previos de P1 y P2.

Comando:

```
docker exec -it cliente python src/cli.py --destino P2 enviar --receptor P3 --msg '{"id":"ORD-  
77X", "pago":"ok", "item":"Super Star"}'
```

5. P3 (Restaurante)

P3 ejecuta un evento interno que lo que hace es que simula la preparación y que se empaqueta la hamburguesa.

Su reloj vectorial avanza en su propia posición.



Comando:

```
docker exec -it cliente python src/cli.py --destino P3 interno --desc "preparar comida"
```

6. P3 → P4 - Aviso al repartidor

P3 envía un mensaje gRPC a P4 indicando que el paquete está listo para recolección.

P4 combina directamente el vector recibido con el suyo, sincronizando su reloj con todos los eventos acumulados.

Comando:

```
docker exec -it cliente python src/cli.py --destino P3 enviar --receptor P4 --msg '{"id":"ORD-77X", "paquete":"listo"}'
```

7. P4 (Repartidor) - Evento interno: calcular ruta GPS

P4 realiza un evento interno que simula el cálculo de la ruta óptima hacia el destino del cliente mediante GPS.

Solo incrementa su posición en el reloj vectorial.

Comando:

```
docker exec -it cliente python src/cli.py --destino P4 interno --desc "calcular ruta gps"
```

8. P4 → P5 – Notificación de inicio de viaje

P4 le comunica a P5 que es el servidor de notificaciones que el repartidor ya está en camino.

P5 recibe el vector con el historial completo de todos los procesos y lo combinad directamente con el suyo para preparar el broadcast.

Comando:

```
docker exec -it cliente python src/cli.py --destino P4 enviar --receptor P5 --msg '"id":"ORD-77X", "repartidor":"va", "eta":"15m"'
```

9. P5 (Servidor de Notificaciones)

P5 ejecuta un evento interno para calcular y registrar el tiempo estimado de llegada antes de mandar el broadcast.

Su reloj vectorial avanza en su propia posición.



Comando:

```
docker exec -it cliente python src/cli.py --destino P5 interno --desc "enviar notificacion"
```

10. P5 – Broadcast final a todos los nodos

P5 envía a todos al mismo tiempo el estado final de la orden a P1, P2, P3 y P4 mediante una difusión gRPC.

Cada proceso receptor compara el vector actualizado de P5 con el suyo y termina con los tiempos actualizados de forma correcta todos.

Comando:

```
docker exec -it cliente python src/cli.py --destino P5 difusion --msg '{"id":"ORD-77X", "fin":"entregado", "eta":"15m"}'
```

5.3. Revision de logs

Para revisar el archivo de bitácora generado por cada nodo al finalizar la secuencia (se muestra el ejemplo para P1, pero aplica para P1 a P5):

```
docker exec -it p1 cat /app/logs/p1.log
```

Para detener los contenedores, esto solo se hace después de matar el proceso del comando de logs, esto baja todos los servicios de docker creados:

```
docker compose down
```




Para ir verificando el desarrollo del proyecto, abrimos una terminal dedicada a mostrar los registros en tiempo real, que sera la terminal superior. Aqui primero comprobamos que los servidores de los procesos ya estan activos y escuchando las peticiones en sus respectivos puertos.

```
rodrico@RodryLinux: ~/Escritorio/FI/8V0/SD/ProyectoFinal/Distribuidos/project/proyecto_final
✓ cliente Built 0.0s
✓ p1 Built 0.0s
✓ p2 Built 0.0s
✓ p3 Built 0.0s
✓ p4 Built 0.0s
✓ p5 Built 0.0s
✓ Network proyecto_final_red_privada Created 0.0s
✓ Container p3 Started 0.4s
✓ Container p1 Started 0.4s
✓ Container p5 Started 0.4s
✓ Container p2 Started 0.4s
✓ Container cliente Started 0.4s
✓ Container p4 Started 0.2s
rodrico@RodryLinux:~/Escritorio/FI/8V0/SD/ProyectoFinal/Distribuidos/project/proyecto_final$ docker compose logs -f
p1 | Nodo P1 activo en el puerto 50051...
p4 | Nodo P4 activo en el puerto 50051...
p2 | Nodo P2 activo en el puerto 50051...
p3 | Nodo P3 activo en el puerto 50051...
p5 | Nodo P5 activo en el puerto 50051...
rodrico@RodryLinux:~/Escritorio/FI/8V0/SD/ProyectoFinal/Distribuidos/project/proyecto_final$
```

Figura 4: Terminal de registros mostrando a los procesos activos.

En una segunda terminal inferior comenzamos a enviar los comandos. El primer paso es mandar un evento interno al Proceso 1 para armar el carrito de comida. En la terminal de registros vemos que el Proceso 1 detecta el evento y su reloj vectorial arranca marcando un uno en la primera posición.

```
rodrico@RodryLinux:~/Escritorio/FI/8V0/SD/ProyectoFinal/Distribuidos/project/proyecto_final
✓ p1 Built 0.0s
✓ p2 Built 0.0s
✓ p3 Built 0.0s
✓ p4 Built 0.0s
✓ p5 Built 0.0s
✓ Network proyecto_final_red_privada Created 0.0s
✓ Container p3 Started 0.4s
✓ Container p1 Started 0.4s
✓ Container p5 Started 0.4s
✓ Container p2 Started 0.4s
✓ Container cliente Started 0.4s
✓ Container p4 Started 0.2s
rodrico@RodryLinux:~/Escritorio/FI/8V0/SD/ProyectoFinal/Distribuidos/project/proyecto_final$ docker compose logs -f
p1 | Nodo P1 activo en el puerto 50051...
p4 | Nodo P4 activo en el puerto 50051...
p2 | Nodo P2 activo en el puerto 50051...
p3 | Nodo P3 activo en el puerto 50051...
p5 | Nodo P5 activo en el puerto 50051...
p1 | [INTERNAL] P1 vector=[1,0,0,0]
rodrico@RodryLinux:~/Escritorio/FI/8V0/SD/ProyectoFinal/Distribuidos/project/proyecto_final$ docker exec -it cliente python src/cli.py --destino P1 interno --desc "armar carrito pedido"
OK: Registro: {"accion":"crear_carrito", "item":"Super Star", "precio":250} (vector=[1,0,0,0])
rodrico@RodryLinux:~/Escritorio/FI/8V0/SD/ProyectoFinal/Distribuidos/project/proyecto_final$
```

Figura 5: Registro del primer evento interno en el reloj vectorial del Proceso 1.

Después le pedimos al Proceso 1 que mande la solicitud de cobro al Proceso 2. El Proceso 2 recibe la informacion de la compra junto con el reloj del emisor. Al comparar los valores, toma el maximo y avanza su propio contador, quedando con el vector actualizado correctamente.



```
des Terminal 1 de jun 23:36
rodrigo@RodyLinux: ~/Escritorio/FI/8VO/SD/ProyectoFinal/Distribuidos/project/proyecto_final
✓ p4 Built 0.0s
✓ p5 Built 0.0s
✓ Network proyecto_final_red_privada Created 0.0s
✓ Container p3 Started 0.4s
✓ Container p1 Started 0.4s
✓ Container p5 Started 0.4s
✓ Container p2 Started 0.4s
✓ Container cliente Started 0.4s
✓ Container p4 Started 0.2s
rodrigo@RodyLinux:~/Escritorio/FI/8VO/SD/ProyectoFinal/Distribuidos/project/proyecto_final$ docker compose logs -f
p1 | Nodo P1 activo en el puerto 50051...
p4 | Nodo P4 activo en el puerto 50051...
p2 | Nodo P2 activo en el puerto 50051...
p3 | Nodo P3 activo en el puerto 50051...
p5 | Nodo P5 activo en el puerto 50051...
p1 | [INTERNAL] P1 vector=[1,0,0,0]
p1 | [SEND] P1 -> P2 msg="{\"id\":\"ORD-77X\", \"total\":250, \"req\":\"cobro\"}" vector r=[2,0,0,0]
p2 | [RECEIVE] P2 <- P1 msg="{\"id\":\"ORD-77X\", \"total\":250, \"req\":\"cobro\"}" vector_recibido=[2,0,0,0] vector_actualizado=[2,1,0,0]
p1 | [ACK] P1 <- P2 confirmacion="ACK" vector_ack=[2,1,0,0]
rodrigo@RodyLinux:~/Escritorio/FI/8VO/SD/ProyectoFinal/Distribuidos/project/proyecto_final$ docker exec -it cliente python src/cli.py --destino P1 inte
rno --desc "armar carrito pedido"
OK: Registro: {\"accion\":\"crear_carrito\", \"item\":\"Super Star\", \"precio\":250} (vector=[1,0,0,0])
rodrigo@RodyLinux:~/Escritorio/FI/8VO/SD/ProyectoFinal/Distribuidos/project/proyecto_final$ docker exec -it cliente python src/cli.py --destino P1 envi
ar --receptor P2 --msg '{\"id\":\"ORD-77X\", \"total\":250, \"req\":\"cobro\"}'
OK: ACK (vector=[2,0,0,0])
rodrigo@RodyLinux:~/Escritorio/FI/8VO/SD/ProyectoFinal/Distribuidos/project/proyecto_final$
```

Figura 6: Envio de la solicitud de cobro y actualizacion del vector en el Proceso 2.

En el siguiente paso ocurren dos cosas. Primero, el Proceso 2 realiza la validación de la tarjeta como una tarea interna, sumando uno a su posición. Inmediatamente después, manda la confirmación del pago al restaurante, que es el Proceso 3. Este último recibe los datos y actualiza su reloj, manteniendo así el orden de todo lo que ha pasado antes.

```
des Terminal 1 de jun 23:37
rodrigo@RodyLinux: ~/Escritorio/FI/8VO/SD/ProyectoFinal/Distribuidos/project/proyecto_final
✓ Container p1 Started 0.4s
✓ Container p5 Started 0.4s
✓ Container p2 Started 0.4s
✓ Container cliente Started 0.4s
✓ Container p4 Started 0.2s
rodrigo@RodyLinux:~/Escritorio/FI/8VO/SD/ProyectoFinal/Distribuidos/project/proyecto_final$ docker compose logs -f
p1 | Nodo P1 activo en el puerto 50051...
p4 | Nodo P4 activo en el puerto 50051...
p2 | Nodo P2 activo en el puerto 50051...
p3 | Nodo P3 activo en el puerto 50051...
p5 | Nodo P5 activo en el puerto 50051...
p1 | [INTERNAL] P1 vector=[1,0,0,0]
p1 | [SEND] P1 -> P2 msg="{\"id\":\"ORD-77X\", \"total\":250, \"req\":\"cobro\"}" vector r=[2,0,0,0]
p2 | [RECEIVE] P2 <- P1 msg="{\"id\":\"ORD-77X\", \"total\":250, \"req\":\"cobro\"}" vector_recibido=[2,0,0,0] vector_actualizado=[2,1,0,0]
p1 | [ACK] P1 <- P2 confirmacion="ACK" vector_ack=[2,1,0,0]
p2 | [INTERNAL] P2 vector=[2,2,0,0]
p2 | [SEND] P2 -> P3 msg="{\"id\":\"ORD-77X\", \"pago\":\"ok\", \"item\":\"Super Star\"}" vector r=[2,3,0,0]
p3 | [RECEIVE] P3 <- P2 msg="{\"id\":\"ORD-77X\", \"pago\":\"ok\", \"item\":\"Super Star\"}" vector_recibido=[2,3,0,0] vector_actualizado=[2,3,1,0]
p2 | [ACK] P2 <- P3 confirmacion="ACK" vector_ack=[2,3,1,0]
rodrigo@RodyLinux:~/Escritorio/FI/8VO/SD/ProyectoFinal/Distribuidos/project/proyecto_final$ docker exec -it cliente python src/cli.py --destino P1 inte
rno --desc "armar carrito pedido"
OK: Registro: {\"accion\":\"crear_carrito\", \"item\":\"Super Star\", \"precio\":250} (vector=[1,0,0,0])
rodrigo@RodyLinux:~/Escritorio/FI/8VO/SD/ProyectoFinal/Distribuidos/project/proyecto_final$ docker exec -it cliente python src/cli.py --destino P1 envi
ar --receptor P2 --msg '{\"id\":\"ORD-77X\", \"total\":250, \"req\":\"cobro\"}'
OK: ACK (vector=[2,0,0,0])
rodrigo@RodyLinux:~/Escritorio/FI/8VO/SD/ProyectoFinal/Distribuidos/project/proyecto_final$ docker exec -it cliente python src/cli.py --destino P2 inte
rno --desc "validar cobro tarjeta"
OK: Registro: {\"accion\":\"validar_tarjeta\", \"status\":\"ok\", \"auth\":\"TXN-9981\"} (vector=[2,2,0,0])
rodrigo@RodyLinux:~/Escritorio/FI/8VO/SD/ProyectoFinal/Distribuidos/project/proyecto_final$ docker exec -it cliente python src/cli.py --destino P2 envi
ar --receptor P3 --msg '{\"id\":\"ORD-77X\", \"pago\":\"ok\", \"item\":\"Super Star\"}'
OK: ACK (vector=[2,3,0,0])
rodrigo@RodyLinux:~/Escritorio/FI/8VO/SD/ProyectoFinal/Distribuidos/project/proyecto_final$
```

Figura 7: Validación del pago en el Proceso 2 y confirmación enviada al Proceso 3.



El restaurante ejecuta de forma local la acción de preparar la comida. Al ser un evento interno, solamente aumenta el contador en la tercera posición de su reloj vectorial.

```
Terminal
rodrigo@RoddyLinux: ~/Escritorio/FI/8VO/SD/ProyectoFinal/Distribuidos/project/proyecto_final$ docker compose logs -f
Container p5 Started 0.4s
Container p2 Started 0.4s
Container cliente Started 0.4s
Container cliente Started 0.2s
rodrigo@RoddyLinux:~/Escritorio/FI/8VO/SD/ProyectoFinal/Distribuidos/project/proyecto_final$ docker exec -it cliente python src/cli.py --destino P1 inte
rno --desc "armar carrito pedido"
OK: Registro: {'accion': 'crear_carrito', 'item': 'Super Star', 'precio': 250} (vector=[1,0,0,0])
rodrigo@RoddyLinux:~/Escritorio/FI/8VO/SD/ProyectoFinal/Distribuidos/project/proyecto_final$ docker exec -it cliente python src/cli.py --destino P1 envi
ar --receptor P2 --msg '{"id": "ORD-77X", "total": 250, "req": "cobre"}'
OK: ACK (vector=[2,0,0,0])
rodrigo@RoddyLinux:~/Escritorio/FI/8VO/SD/ProyectoFinal/Distribuidos/project/proyecto_final$ docker exec -it cliente python src/cli.py --destino P2 inte
rno --desc "validar cobro tarjeta"
OK: Registro: {'accion': 'validar_tarjeta', 'status': 'ok', 'auth': 'TXN-9981'} (vector=[2,2,0,0])
rodrigo@RoddyLinux:~/Escritorio/FI/8VO/SD/ProyectoFinal/Distribuidos/project/proyecto_final$ docker exec -it cliente python src/cli.py --destino P2 envi
ar --receptor P3 --msg '{"id": "ORD-77X", "pago": "ok", "item": "Super Star"}'
OK: ACK (vector=[2,3,0,0])
rodrigo@RoddyLinux:~/Escritorio/FI/8VO/SD/ProyectoFinal/Distribuidos/project/proyecto_final$ docker exec -it cliente python src/cli.py --destino P3 inte
rno --desc "preparar comida"
OK: Registro: {'accion': 'cocina', 'status': 'empaquetado', 'temp': 'caliente'} (vector=[2,3,2,0])
rodrigo@RoddyLinux:~/Escritorio/FI/8VO/SD/ProyectoFinal/Distribuidos/project/proyecto_final$
```

Figura 8: Evento interno de preparar la comida en el Proceso 3.

Cuando el paquete esta listo, el restaurante manda el aviso al repartidor, asignado al Proceso 4. El repartidor toma el mensaje, aplica la regla de los maximos al comparar los vectores y ajusta su reloj para estar sincronizado con el resto de la cadena.

```
Terminal
rodrigo@RoddyLinux: ~/Escritorio/FI/8VO/SD/ProyectoFinal/Distribuidos/project/proyecto_final$ docker compose logs -f
Container p4 Started 0.2s
rodrigo@RoddyLinux:~/Escritorio/FI/8VO/SD/ProyectoFinal/Distribuidos/project/proyecto_final$ docker exec -it cliente python src/cli.py --destino P1 inte
rno --desc "armar carrito pedido"
OK: Registro: {'accion': 'crear_carrito', 'item': 'Super Star', 'precio': 250} (vector=[1,0,0,0])
rodrigo@RoddyLinux:~/Escritorio/FI/8VO/SD/ProyectoFinal/Distribuidos/project/proyecto_final$ docker exec -it cliente python src/cli.py --destino P1 envi
ar --receptor P2 --msg '{"id": "ORD-77X", "total": 250, "req": "cobre"}'
OK: ACK (vector=[2,0,0,0])
rodrigo@RoddyLinux:~/Escritorio/FI/8VO/SD/ProyectoFinal/Distribuidos/project/proyecto_final$ docker exec -it cliente python src/cli.py --destino P2 inte
rno --desc "validar cobro tarjeta"
OK: Registro: {'accion': 'validar_tarjeta', 'status': 'ok', 'auth': 'TXN-9981'} (vector=[2,2,0,0])
rodrigo@RoddyLinux:~/Escritorio/FI/8VO/SD/ProyectoFinal/Distribuidos/project/proyecto_final$ docker exec -it cliente python src/cli.py --destino P2 envi
ar --receptor P3 --msg '{"id": "ORD-77X", "pago": "ok", "item": "Super Star"}'
OK: ACK (vector=[2,3,0,0])
rodrigo@RoddyLinux:~/Escritorio/FI/8VO/SD/ProyectoFinal/Distribuidos/project/proyecto_final$ docker exec -it cliente python src/cli.py --destino P3 inte
rno --desc "preparar comida"
OK: Registro: {'accion': 'cocina', 'status': 'empaquetado', 'temp': 'caliente'} (vector=[2,3,2,0])
rodrigo@RoddyLinux:~/Escritorio/FI/8VO/SD/ProyectoFinal/Distribuidos/project/proyecto_final$ docker exec -it cliente python src/cli.py --destino P3 envi
ar --receptor P4 --msg '{"id": "ORD-77X", "paquete": "listo"}'
OK: ACK (vector=[2,3,3,0])
rodrigo@RoddyLinux:~/Escritorio/FI/8VO/SD/ProyectoFinal/Distribuidos/project/proyecto_final$
```

Figura 9: Aviso al repartidor y sincronización de su reloj vectorial.



El repartidor calcula la ruta de entrega mediante un evento interno. La consola nos muestra como su contador sube un numero en la cuarta posicion.

```
rodrigo@RodrigoLinux: ~/Escritorio/FI/8VO/SD/ProyectoFinal/Distribuidos/project/proyecto_final$ docker compose logs -f
p1 | Nodo P1 activo en el puerto 50051...
p4 | Nodo P4 activo en el puerto 50051...
p2 | Nodo P2 activo en el puerto 50051...
p3 | Nodo P3 activo en el puerto 50051...
p5 | Nodo P5 activo en el puerto 50051...
p1 | [INTERNAL] P1 vector=[1,0,0,0,0]
p4 | [SEND] P1 -> P2 msg="{\"id\":\"ORD-77X\", \"total\":250, \"req\":\"cobro\"}" vector r=[2,0,0,0,0]
p2 | [RECEIVE] P2 <- P1 msg="{\"id\":\"ORD-77X\", \"total\":250, \"req\":\"cobro\"}" vector_recibido=[2,0,0,0,0] vector_actualizado=[2,1,0,0,0]
p1 | [ACK] P1 <- P2 confirmacion="ACK" vector_ack=[2,1,0,0,0]
p2 | [INTERNAL] P2 vector=[2,2,0,0,0]
p2 | [SEND] P2 -> P3 msg="{\"id\":\"ORD-77X\", \"pago\":\"ok\", \"item\":\"Super Star\"}" vector r=[2,3,0,0,0]
p3 | [RECEIVE] P3 <- P2 msg="{\"id\":\"ORD-77X\", \"pago\":\"ok\", \"item\":\"Super Star\"}" vector_recibido=[2,3,0,0,0] vector_actualizado=[2,3,1,0,0]
p2 | [ACK] P2 <- P3 confirmacion="ACK" vector_ack=[2,3,1,0,0]
p3 | [INTERNAL] P3 vector=[2,3,2,0,0]
p3 | [SEND] P3 -> P4 msg="{\"id\":\"ORD-77X\", \"paquete\":\"listo\"}" vector r=[2,3,3,0,0]
p4 | [RECEIVE] P4 <- P3 msg="{\"id\":\"ORD-77X\", \"paquete\":\"listo\"}" vector_recibido=[2,3,3,0,0] vector_actualizado=[2,3,3,1,0]
p3 | [ACK] P3 <- P4 confirmacion="ACK" vector_ack=[2,3,3,1,0]
p4 | [INTERNAL] P4 vector=[2,3,3,2,0]

rodrigo@RodrigoLinux: ~/Escritorio/FI/8VO/SD/ProyectoFinal/Distribuidos/project/proyecto_final$
OK: ACK (vector=[2,0,0,0,0])
rodrigo@RodrigoLinux: ~/Escritorio/FI/8VO/SD/ProyectoFinal/Distribuidos/project/proyecto_final$ docker exec -it cliente python src/cli.py --destino P2 inte
rno --desc "validar cobro tarjeta"
OK: Registro: {"accion":"validar_tarjeta", "status":"ok", "auth":"TXN-9981"} (vector=[2,2,0,0,0])
rodrigo@RodrigoLinux: ~/Escritorio/FI/8VO/SD/ProyectoFinal/Distribuidos/project/proyecto_final$ docker exec -it cliente python src/cli.py --destino P2 envi
ar --receptor P3 --msg '{"id":"ORD-77X", "pago":"ok", "item":"Super Star"}'
OK: ACK (vector=[2,3,0,0,0])
rodrigo@RodrigoLinux: ~/Escritorio/FI/8VO/SD/ProyectoFinal/Distribuidos/project/proyecto_final$ docker exec -it cliente python src/cli.py --destino P3 inte
rno --desc "preparar comida"
OK: Registro: {"accion":"cocina", "status":"empaquetado", "temp":"caliente"} (vector=[2,3,2,0,0])
rodrigo@RodrigoLinux: ~/Escritorio/FI/8VO/SD/ProyectoFinal/Distribuidos/project/proyecto_final$ docker exec -it cliente python src/cli.py --destino P3 envi
ar --receptor P4 --msg '{"id":"ORD-77X", "paquete":"listo"}'
OK: ACK (vector=[2,3,3,0,0])
rodrigo@RodrigoLinux: ~/Escritorio/FI/8VO/SD/ProyectoFinal/Distribuidos/project/proyecto_final$ docker exec -it cliente python src/cli.py --destino P4 inte
rno --desc "calcular ruta gps"
OK: Registro: {"accion":"gps", "destino":"Tlalnepanitla", "dist":"4.2km"} (vector=[2,3,3,2,0])
rodrigo@RodrigoLinux: ~/Escritorio/FI/8VO/SD/ProyectoFinal/Distribuidos/project/proyecto_final$
```

Figura 10: Calculo de la ruta como evento interno en el Proceso 4.

Al iniciar el recorrido, el repartidor le avisa al servidor de notificaciones, que es el Proceso 5. El servidor recibe el vector acumulado y actualiza la quinta posicion de su reloj.

```
rodrigo@RodrigoLinux: ~/Escritorio/FI/8VO/SD/ProyectoFinal/Distribuidos/project/proyecto_final$
p2 | Nodo P2 activo en el puerto 50051...
p3 | Nodo P3 activo en el puerto 50051...
p5 | Nodo P5 activo en el puerto 50051...
p1 | [INTERNAL] P1 vector=[1,0,0,0,0]
p4 | [SEND] P1 -> P2 msg="{\"id\":\"ORD-77X\", \"total\":250, \"req\":\"cobro\"}" vector r=[2,0,0,0,0]
p2 | [RECEIVE] P2 <- P1 msg="{\"id\":\"ORD-77X\", \"total\":250, \"req\":\"cobro\"}" vector_recibido=[2,0,0,0,0] vector_actualizado=[2,1,0,0,0]
p1 | [ACK] P1 <- P2 confirmacion="ACK" vector_ack=[2,1,0,0,0]
p2 | [INTERNAL] P2 vector=[2,2,0,0,0]
p2 | [SEND] P2 -> P3 msg="{\"id\":\"ORD-77X\", \"pago\":\"ok\", \"item\":\"Super Star\"}" vector r=[2,3,0,0,0]
p3 | [RECEIVE] P3 <- P2 msg="{\"id\":\"ORD-77X\", \"pago\":\"ok\", \"item\":\"Super Star\"}" vector_recibido=[2,3,0,0,0] vector_actualizado=[2,3,1,0,0]
p2 | [ACK] P2 <- P3 confirmacion="ACK" vector_ack=[2,3,1,0,0]
p3 | [INTERNAL] P3 vector=[2,3,2,0,0]
p3 | [SEND] P3 -> P4 msg="{\"id\":\"ORD-77X\", \"paquete\":\"listo\"}" vector r=[2,3,3,0,0]
p4 | [RECEIVE] P4 <- P3 msg="{\"id\":\"ORD-77X\", \"paquete\":\"listo\"}" vector_recibido=[2,3,3,0,0] vector_actualizado=[2,3,3,1,0]
p3 | [ACK] P3 <- P4 confirmacion="ACK" vector_ack=[2,3,3,1,0]
p4 | [INTERNAL] P4 vector=[2,3,3,2,0]
p4 | [SEND] P4 -> P5 msg="{\"id\":\"ORD-77X\", \"repartidor\":\"va\", \"eta\":\"15m\"}" vector r=[2,3,3,3,0]
p5 | [RECEIVE] P5 <- P4 msg="{\"id\":\"ORD-77X\", \"repartidor\":\"va\", \"eta\":\"15m\"}" vector_recibido=[2,3,3,3,0] vector_actualizado=[2,3,3,3,1]
p4 | [ACK] P4 <- P5 confirmacion="ACK" vector_ack=[2,3,3,3,1]

rodrigo@RodrigoLinux: ~/Escritorio/FI/8VO/SD/ProyectoFinal/Distribuidos/project/proyecto_final$
OK: Registro: {"accion":"validar_tarjeta", "status":"ok", "auth":"TXN-9981"} (vector=[2,2,0,0,0])
rodrigo@RodrigoLinux: ~/Escritorio/FI/8VO/SD/ProyectoFinal/Distribuidos/project/proyecto_final$ docker exec -it cliente python src/cli.py --destino P2 envi
ar --receptor P3 --msg '{"id":"ORD-77X", "pago":"ok", "item":"Super Star"}'
OK: ACK (vector=[2,3,0,0,0])
rodrigo@RodrigoLinux: ~/Escritorio/FI/8VO/SD/ProyectoFinal/Distribuidos/project/proyecto_final$ docker exec -it cliente python src/cli.py --destino P3 inte
rno --desc "preparar comida"
OK: Registro: {"accion":"cocina", "status":"empaquetado", "temp":"caliente"} (vector=[2,3,2,0,0])
rodrigo@RodrigoLinux: ~/Escritorio/FI/8VO/SD/ProyectoFinal/Distribuidos/project/proyecto_final$ docker exec -it cliente python src/cli.py --destino P3 envi
ar --receptor P4 --msg '{"id":"ORD-77X", "paquete":"listo"}'
OK: ACK (vector=[2,3,3,0,0])
rodrigo@RodrigoLinux: ~/Escritorio/FI/8VO/SD/ProyectoFinal/Distribuidos/project/proyecto_final$ docker exec -it cliente python src/cli.py --destino P4 inte
rno --desc "calcular ruta gps"
OK: Registro: {"accion":"gps", "destino":"Tlalnepanitla", "dist":"4.2km"} (vector=[2,3,3,2,0])
rodrigo@RodrigoLinux: ~/Escritorio/FI/8VO/SD/ProyectoFinal/Distribuidos/project/proyecto_final$ docker exec -it cliente python src/cli.py --destino P4 envi
ar --receptor P5 --msg '{"id":"ORD-77X", "repartidor":"va", "eta":"15m"}'
OK: ACK (vector=[2,3,3,3,0])
rodrigo@RodrigoLinux: ~/Escritorio/FI/8VO/SD/ProyectoFinal/Distribuidos/project/proyecto_final$
```

Figura 11: Notificacion de viaje al servidor central y actualizacion de su vector.



```

p3 | Nodo P3 activo en el puerto 50051...
p5 | Nodo P5 activo en el puerto 50051...
p1 | [INTERNAL] P1 vector=[1,0,0,0]
p1 | [SEND] P1 -> P2 msg="{'id':'ORD-77X', 'total':250, 'req':'cobro'}" vector r=[2,0,0,0]
p1 | [RECEIVE] P2 <- P1 msg="{'id':'ORD-77X', 'total':250, 'req':'cobro'}" vector_recibido=[2,0,0,0] vector_actualizado=[2,1,0,0]
p1 | [ACK] P1 <- P2 confirmacion="ACK" vector_ack=[2,1,0,0]
p1 | [INTERNAL] P2 vector=[2,2,0,0]
p1 | [SEND] P2 -> P3 msg="{'id':'ORD-77X', 'pago':'ok', 'item':'Super Star'}" vector r=[2,3,0,0]
p1 | [RECEIVE] P3 <- P2 msg="{'id':'ORD-77X', 'pago':'ok', 'item':'Super Star'}" vector_recibido=[2,3,0,0] vector_actualizado=[2,3,1,0]
p1 | [ACK] P2 <- P3 confirmacion="ACK" vector_ack=[2,3,1,0]
p1 | [INTERNAL] P3 vector=[2,3,2,0]
p1 | [SEND] P3 -> P4 msg="{'id':'ORD-77X', 'package':'listo'}" vector r=[2,3,3,0]
p1 | [RECEIVE] P4 <- P3 msg="{'id':'ORD-77X', 'package':'listo'}" vector_recibido=[2,3,3,0] vector_actualizado=[2,3,3,1]
p1 | [ACK] P3 <- P4 confirmacion="ACK" vector_ack=[2,3,3,1]
p1 | [INTERNAL] P4 vector=[2,3,3,2]
p1 | [SEND] P4 -> P5 msg="{'id':'ORD-77X', 'repartidor':'va', 'eta':'15m'}" vector r=[2,3,3,3]
p1 | [RECEIVE] P5 <- P4 msg="{'id':'ORD-77X', 'repartidor':'va', 'eta':'15m'}" vector_recibido=[2,3,3,3] vector_actualizado=[2,3,3,3]
p1 | [ACK] P4 <- P5 confirmacion="ACK" vector_ack=[2,3,3,3]
p1 | [INTERNAL] P5 vector=[2,3,3,2]
p1 |

```

```

rodrigo@RodryLinux: ~/Escritorio/FI/BVQ/SD/ProyectoFinal/Distribuidos/project/proyecto_final
OK: ACK (vector=[2,3,0,0])
rodrigo@RodryLinux: ~/Escritorio/FI/BVQ/SD/ProyectoFinal/Distribuidos/project/proyecto_final$ docker exec -it cliente python src/cli.py --destino P3 inte
rno -desc "preparar comida"
OK: Registro: ('accion': 'cocina', 'status': 'empaquetado', 'temp': 'caliente') (vector=[2,3,2,0])
rodrigo@RodryLinux: ~/Escritorio/FI/BVQ/SD/ProyectoFinal/Distribuidos/project/proyecto_final$ docker exec -it cliente python src/cli.py --destino P3 envi
ar --receptor P4 -msg {'id':'ORD-77X', 'package':'listo'}
OK: ACK (vector=[2,3,3,0])
rodrigo@RodryLinux: ~/Escritorio/FI/BVQ/SD/ProyectoFinal/Distribuidos/project/proyecto_final$ docker exec -it cliente python src/cli.py --destino P4 inte
rno -desc "calcular ruta gps"
OK: Registro: ('accion': 'gps', 'destino': 'Itnepantla', 'dist': '4.2km') (vector=[2,3,3,0])
rodrigo@RodryLinux: ~/Escritorio/FI/BVQ/SD/ProyectoFinal/Distribuidos/project/proyecto_final$ docker exec -it cliente python src/cli.py --destino P4 envi
ar --receptor P5 -msg {'id':'ORD-77X', 'repartidor':'va', 'eta':'15m'}
OK: ACK (vector=[2,3,3,3])
rodrigo@RodryLinux: ~/Escritorio/FI/BVQ/SD/ProyectoFinal/Distribuidos/project/proyecto_final$ docker exec -it cliente python src/cli.py --destino P5 inte
rno -desc "enviar notificacion"
OK: Registro: ('accion': 'push', 'disp': 'Celular', 'status': 'enviado') (vector=[2,3,3,2])
rodrigo@RodryLinux: ~/Escritorio/FI/BVQ/SD/ProyectoFinal/Distribuidos/project/proyecto_final$

```

Finalmente llegamos a la difusion. El servidor manda el aviso de entrega a todos los procesos al mismo tiempo. Cada proceso recibe el vector final y, al actualizar sus posiciones, todos los relojes terminan exactamente iguales. Esto demuestra que se logro la sincronizacion de todo el sistema.

```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000

```

30



Para concluir, ejecutamos el comando para apagar los servicios. Esto detiene los contenedores, los elimina y borra la red privada, dejando el entorno de trabajo completamente limpio.

```
rodriago@RodrigoLinux: ~/Escritorio/FI/8VO/SD/ProyectoFinal/Distribuidos/project/proyecto_final
p5 [INTERNAL] P5 vector=[2,3,3,3,2]
p5 [SEND] P5 -> P1 msg="{\"id\":\"ORD-77X\", \"fin\":\"entregado\", \"eta\":\"15m\"}" vector r=[2,3,3,3,3]
p1 [RECEIVE] P1 <- P5 msg="{\"id\":\"ORD-77X\", \"fin\":\"entregado\", \"eta\":\"15m\"}" vector_recibido=[2,3,3,3,3] vector_actualizado=[3,3,3,3,3]
p5 [SEND] P5 -> P2 msg="{\"id\":\"ORD-77X\", \"fin\":\"entregado\", \"eta\":\"15m\"}" vector r=[2,3,3,3,3]
p2 [RECEIVE] P2 <- P5 msg="{\"id\":\"ORD-77X\", \"fin\":\"entregado\", \"eta\":\"15m\"}" vector_recibido=[2,3,3,3,3] vector_actualizado=[2,4,3,3,3]
p5 [SEND] P5 -> P3 msg="{\"id\":\"ORD-77X\", \"fin\":\"entregado\", \"eta\":\"15m\"}" vector r=[2,3,3,3,3]
p3 [RECEIVE] P3 <- P5 msg="{\"id\":\"ORD-77X\", \"fin\":\"entregado\", \"eta\":\"15m\"}" vector_recibido=[2,3,3,3,3] vector_actualizado=[2,3,4,3,3]
p5 [SEND] P5 -> P4 msg="{\"id\":\"ORD-77X\", \"fin\":\"entregado\", \"eta\":\"15m\"}" vector r=[2,3,3,3,3]
p4 [RECEIVE] P4 <- P5 msg="{\"id\":\"ORD-77X\", \"fin\":\"entregado\", \"eta\":\"15m\"}" vector_recibido=[2,3,3,3,3] vector_actualizado=[2,3,3,4,3]
AC

rodriago@RodrigoLinux: ~/Escritorio/FI/8VO/SD/ProyectoFinal/Distribuidos/project/proyecto_final$ docker compose down
[+] Running 7/7
  Container p3              Removed                               11.0s
  Container p5              Removed                               10.6s
  Container p4              Removed                               10.9s
  Container p1              Removed                               10.8s
  Container p2              Removed                               10.7s
  Container cliente         Removed                               10.5s
  Network proyecto_final_red_privada Removed                               0.1s

rodriago@RodrigoLinux: ~/Escritorio/FI/8VO/SD/ProyectoFinal/Distribuidos/project/proyecto_final$
[SEND] P3 -> P4 msg="{\"id\":\"ORD-77X\", \"paquete\":\"listo\"}" vector r=[2,3,3,0,0]
[ACK] P3 <- P4 confirmacion="ACK" vector_ack=[2,3,3,1,0]
[RECEIVE] P3 <- P5 msg="{\"id\":\"ORD-77X\", \"fin\":\"entregado\", \"eta\":\"15m\"}" vector_recibido=[2,3,3,3,3] vector_actualizado=[2,3,4,3,3]
rodriago@RodrigoLinux: ~/Escritorio/FI/8VO/SD/ProyectoFinal/Distribuidos/project/proyecto_final$ docker exec -it p4 cat /app/logs/p4.log
[RECEIVE] P4 <- P3 msg="{\"id\":\"ORD-77X\", \"paquete\":\"listo\"}" vector_recibido=[2,3,3,0,0] vector_actualizado=[2,3,3,1,0]
[INTERNAL] P4 vector=[2,3,3,2,0]
[SEND] P4 -> P5 msg="{\"id\":\"ORD-77X\", \"repartidor\":\"va\", \"eta\":\"15m\"}" vector r=[2,3,3,3,0]
[ACK] P4 <- P5 confirmacion="ACK" vector_ack=[2,3,3,3,1]
[RECEIVE] P4 <- P5 msg="{\"id\":\"ORD-77X\", \"fin\":\"entregado\", \"eta\":\"15m\"}" vector_recibido=[2,3,3,3,3] vector_actualizado=[2,3,3,4,3]
rodriago@RodrigoLinux: ~/Escritorio/FI/8VO/SD/ProyectoFinal/Distribuidos/project/proyecto_final$ docker exec -it p5 cat /app/logs/p5.log
[RECEIVE] P5 <- P4 msg="{\"id\":\"ORD-77X\", \"repartidor\":\"va\", \"eta\":\"15m\"}" vector_recibido=[2,3,3,3,0] vector_actualizado=[2,3,3,3,1]
[INTERNAL] P5 vector=[2,3,3,3,2]
[SEND] P5 -> P1 msg="{\"id\":\"ORD-77X\", \"fin\":\"entregado\", \"eta\":\"15m\"}" vector r=[2,3,3,3,3]
[SEND] P5 -> P2 msg="{\"id\":\"ORD-77X\", \"fin\":\"entregado\", \"eta\":\"15m\"}" vector r=[2,3,3,3,3]
[SEND] P5 -> P3 msg="{\"id\":\"ORD-77X\", \"fin\":\"entregado\", \"eta\":\"15m\"}" vector r=[2,3,3,3,3]
[SEND] P5 -> P4 msg="{\"id\":\"ORD-77X\", \"fin\":\"entregado\", \"eta\":\"15m\"}" vector r=[2,3,3,3,3]
rodriago@RodrigoLinux: ~/Escritorio/FI/8VO/SD/ProyectoFinal/Distribuidos/project/proyecto_final$
```

Figura 14: Limpieza del entorno deteniendo y borrando los contenedores.

6.2. Bitacoras

Para verificar que la historia de los eventos quedo guardada, consultamos los archivos de texto que cada contenedor genero durante la prueba. En las siguientes capturas se observa la lectura directa de estos archivos desde la línea de comandos, comprobando que la informacion si se queda almacenada incluso despues de que los procesos terminan sus tareas.

```
rodriago@RodrigoLinux: ~/Escritorio/FI/8VO/SD/ProyectoFinal/Distribuidos/project/proyecto_final
rodriago@RodrigoLinux: ~/Escritorio/FI/8VO/SD/ProyectoFinal/Distribuidos/project/proyecto_final$ docker exec -it p1 cat /app/logs/p1.log
[INTERNAL] P1 vector=[1,0,0,0,0]
[SEND] P1 -> P2 msg="{\"id\":\"ORD-77X\", \"total\":250, \"req\":\"cobro\"}" vector r=[2,0,0,0,0]
[ACK] P1 <- P2 confirmacion="ACK" vector_ack=[2,1,0,0,0]
[RECEIVE] P1 <- P5 msg="{\"id\":\"ORD-77X\", \"fin\":\"entregado\", \"eta\":\"15m\"}" vector_recibido=[2,3,3,3,3] vector_actualizado=[3,3,3,3,3]
rodriago@RodrigoLinux: ~/Escritorio/FI/8VO/SD/ProyectoFinal/Distribuidos/project/proyecto_final$ docker exec -it p2 cat /app/logs/p2.log
[RECEIVE] P2 <- P1 msg="{\"id\":\"ORD-77X\", \"total\":250, \"req\":\"cobro\"}" vector_recibido=[2,0,0,0,0] vector_actualizado=[2,1,0,0,0]
[INTERNAL] P2 vector=[2,2,0,0,0]
[SEND] P2 -> P3 msg="{\"id\":\"ORD-77X\", \"pago\":\"ok\", \"item\":\"Super Star\"}" vector r=[2,3,0,0,0]
[ACK] P2 <- P3 confirmacion="ACK" vector_ack=[2,3,1,0,0]
[RECEIVE] P2 <- P5 msg="{\"id\":\"ORD-77X\", \"fin\":\"entregado\", \"eta\":\"15m\"}" vector_recibido=[2,3,3,3,3] vector_actualizado=[2,4,3,3,3]
rodriago@RodrigoLinux: ~/Escritorio/FI/8VO/SD/ProyectoFinal/Distribuidos/project/proyecto_final$ docker exec -it p3 cat /app/logs/p3.log
[RECEIVE] P3 <- P2 msg="{\"id\":\"ORD-77X\", \"pago\":\"ok\", \"item\":\"Super Star\"}" vector_recibido=[2,3,0,0,0] vector_actualizado=[2,3,1,0,0]
[INTERNAL] P3 vector=[2,3,2,0,0]
[SEND] P3 -> P4 msg="{\"id\":\"ORD-77X\", \"paquete\":\"listo\"}" vector r=[2,3,3,0,0]
[ACK] P3 <- P4 confirmacion="ACK" vector_ack=[2,3,3,1,0]
[RECEIVE] P3 <- P5 msg="{\"id\":\"ORD-77X\", \"fin\":\"entregado\", \"eta\":\"15m\"}" vector_recibido=[2,3,3,3,3] vector_actualizado=[2,3,4,3,3]
```

Figura 15: Lectura en terminal de las bitacoras de los tres primeros procesos.



```
rodrigo@RodyLinux: ~/Escritorio/FI/8VO/SD/ProyectoFinal/Distribuidos/project/proyecto_final
[SEND] P3 -> P4 msg="{\"id\":\"ORD-77X\", \"paquete\":\"listo\"}" vector r=[2,3,3,0,0]
[ACK] P3 <- P4 confirmacion="ACK" vector_ack=[2,3,3,1,0]
[RECEIVE] P3 <- P5 msg="{\"id\":\"ORD-77X\", \"fin\":\"entregado\", \"eta\":\"15m\"}" vector_recibido=[2,3,3,3,3] vector_actualizado=[2,3,4,3,3]
rodrigo@RodyLinux:~/Escritorio/FI/8VO/SD/ProyectoFinal/Distribuidos/project/proyecto_final$ docker exec -it p4 cat /app/logs/p4.log
[RECEIVE] P4 <- P3 msg="{\"id\":\"ORD-77X\", \"paquete\":\"listo\"}" vector_recibido=[2,3,3,0,0] vector_actualizado=[2,3,3,1,0]
[INTERNAL] P4 vector=[2,3,3,2,0]
[SEND] P4 -> P5 msg="{\"id\":\"ORD-77X\", \"fin\":\"entregado\", \"eta\":\"15m\"}" vector r=[2,3,3,3,0]
[ACK] P4 <- P5 confirmacion="ACK" vector_ack=[2,3,3,3,1]
[RECEIVE] P4 <- P5 msg="{\"id\":\"ORD-77X\", \"fin\":\"entregado\", \"eta\":\"15m\"}" vector_recibido=[2,3,3,3,3] vector_actualizado=[2,3,3,4,3]
rodrigo@RodyLinux:~/Escritorio/FI/8VO/SD/ProyectoFinal/Distribuidos/project/proyecto_final$ docker exec -it p5 cat /app/logs/p5.log
[RECEIVE] P5 <- P4 msg="{\"id\":\"ORD-77X\", \"repartidor\":\"va\", \"eta\":\"15m\"}" vector_recibido=[2,3,3,3,0] vector_actualizado=[2,3,3,3,1]
[INTERNAL] P5 vector=[2,3,3,3,2]
[SEND] P5 -> P1 msg="{\"id\":\"ORD-77X\", \"fin\":\"entregado\", \"eta\":\"15m\"}" vector r=[2,3,3,3,3]
[SEND] P5 -> P2 msg="{\"id\":\"ORD-77X\", \"fin\":\"entregado\", \"eta\":\"15m\"}" vector r=[2,3,3,3,3]
[SEND] P5 -> P3 msg="{\"id\":\"ORD-77X\", \"fin\":\"entregado\", \"eta\":\"15m\"}" vector r=[2,3,3,3,3]
[SEND] P5 -> P4 msg="{\"id\":\"ORD-77X\", \"fin\":\"entregado\", \"eta\":\"15m\"}" vector r=[2,3,3,3,3]
rodrigo@RodyLinux:~/Escritorio/FI/8VO/SD/ProyectoFinal/Distribuidos/project/proyecto_final$
```

Figura 16: Lectura en terminal de las bitacoras del repartidor y el servidor.

A continuacion detallamos el contenido exacto de cada bitacora extraída de los contenedores. En el archivo del Proceso 1 podemos ver como inicio la orden con el evento interno, el envio de la petición de cobro y como al final recibió la difusión del servidor, actualizando su reloj vectorial para igualar el estado final de toda la red.

```
[INTERNAL] P1 vector=[1,0,0,0,0]
[SEND] P1 -> P2 msg="{\"id\":\"ORD-77X\", \"total\":250, \"req\":\"cobro\"}" vector r
    =[2,0,0,0,0]
[ACK] P1 <- P2 confirmacion="ACK" vector_ack=[2,1,0,0,0]
[RECEIVE] P1 <- P5 msg="{\"id\":\"ORD-77X\", \"fin\":\"entregado\", \"eta\":\"15m\"}"
    vector_recibido=[2,3,3,3,3] vector_actualizado=[3,3,3,3,3]
```

En el registro del Proceso 2 se observa la recepción puntual del cobro, su evento interno de validación bancaria y el envío de la orden hacia la cocina del restaurante. El archivo cierra con el mensaje final del servidor de notificaciones.

```
[RECEIVE] P2 <- P1 msg="{\"id\":\"ORD-77X\", \"total\":250, \"req\":\"cobro\"}"
    vector_recibido=[2,0,0,0,0] vector_actualizado=[2,1,0,0,0]
[INTERNAL] P2 vector=[2,2,0,0,0]
[SEND] P2 -> P3 msg="{\"id\":\"ORD-77X\", \"pago\":\"ok\", \"item\":\"Super Star\"}" vector r
    =[2,3,0,0,0]
[ACK] P2 <- P3 confirmacion="ACK" vector_ack=[2,3,1,0,0]
[RECEIVE] P2 <- P5 msg="{\"id\":\"ORD-77X\", \"fin\":\"entregado\", \"eta\":\"15m\"}"
    vector_recibido=[2,3,3,3,3] vector_actualizado=[2,4,3,3,3]
```

La bitacora del Proceso 3 muestra como el restaurante recibe la confirmacion de pago aprobada, ejecuta su evento para preparar la comida y le avisa al repartidor que el paquete esta listo para ser recogido, manteniendo la suma de los vectores en orden.

```
[RECEIVE] P3 <- P2 msg="{\"id\":\"ORD-77X\", \"pago\":\"ok\", \"item\":\"Super Star\"}"
    vector_recibido=[2,3,0,0,0] vector_actualizado=[2,3,1,0,0]
```



```
[INTERNAL] P3 vector=[2,3,2,0,0]
[SEND] P3 -> P4 msg="{\"id\":\"ORD-77X\", \"paquete\":\"listo\"}" vector r=[2,3,3,0,0]
[ACK] P3 <- P4 confirmacion="ACK" vector_ack=[2,3,3,1,0]
[RECEIVE] P3 <- P5 msg="{\"id\":\"ORD-77X\", \"fin\":\"entregado\", \"eta\":\"15m\"}"
vector_recibido=[2,3,3,3,3] vector_actualizado=[2,3,4,3,3]
```

El archivo del Proceso 4 refleja unicamente la interacción del repartidor. Aquí se guarda la recepción del aviso emitido por el restaurante, el evento interno para calcular la ruta geográfica y la notificación que se manda al servidor central para indicar el inicio del viaje.

```
[RECEIVE] P4 <- P3 msg="{\"id\":\"ORD-77X\", \"paquete\":\"listo\"}" vector_recibido
=[2,3,3,0,0] vector_actualizado=[2,3,3,1,0]
[INTERNAL] P4 vector=[2,3,3,2,0]
[SEND] P4 -> P5 msg="{\"id\":\"ORD-77X\", \"repartidor\":\"va\", \"eta\":\"15m\"}" vector r
=[2,3,3,3,0]
[ACK] P4 <- P5 confirmacion="ACK" vector_ack=[2,3,3,3,1]
[RECEIVE] P4 <- P5 msg="{\"id\":\"ORD-77X\", \"fin\":\"entregado\", \"eta\":\"15m\"}"
vector_recibido=[2,3,3,3,3] vector_actualizado=[2,3,3,4,3]
```

Al último en el registro del Proceso 5 se documenta la llegada del aviso del repartidor y el calculo interno del tiempo estimado de llegada. La bitacora concluye mostrando los cuatro mensajes de difusión que el servidor envió simultaneamente al resto de los procesos para cerrar las operaciones.

```
[RECEIVE] P5 <- P4 msg="{\"id\":\"ORD-77X\", \"repartidor\":\"va\", \"eta\":\"15m\"}"
vector_recibido=[2,3,3,3,0] vector_actualizado=[2,3,3,3,1]
[INTERNAL] P5 vector=[2,3,3,3,2]
[SEND] P5 -> P1 msg="{\"id\":\"ORD-77X\", \"fin\":\"entregado\", \"eta\":\"15m\"}" vector r
=[2,3,3,3,3]
[SEND] P5 -> P2 msg="{\"id\":\"ORD-77X\", \"fin\":\"entregado\", \"eta\":\"15m\"}" vector r
=[2,3,3,3,3]
[SEND] P5 -> P3 msg="{\"id\":\"ORD-77X\", \"fin\":\"entregado\", \"eta\":\"15m\"}" vector r
=[2,3,3,3,3]
[SEND] P5 -> P4 msg="{\"id\":\"ORD-77X\", \"fin\":\"entregado\", \"eta\":\"15m\"}" vector r
=[2,3,3,3,3]
```



7. Conclusión

Realizar este proyecto nos permitió agrupar varios de los conceptos teóricos y prácticos que manejamos sobre los sistemas distribuidos y revisamos durante el curso.

El uso de los contenedores de Docker fue muy importante, ya que nos ayudó a separar el entorno de cada proceso, asegurandonos que todos trabajaran de manera independiente dentro de una misma red privada. Por otro lado, trabajar con los archivos proto y la arquitectura de gRPC nos mostro una vez más lo eficiente que puede ser la comunicación entre distintos microservicios, permitiendo enviar y recibir mensajes estructurados de forma muy rápida.

También uno de los mayores aprendizajes del proyecto se dio al momento de programar y observar el comportamiento de los relojes vectoriales. A través de nuestro escenario de pedidos de comida, pudimos comprobar cómo cada proceso actualizaba su propio arreglo numérico al comparar y tomar los valores máximos de los mensajes que recibía. Por otra parte, las bitácoras nos demostraron que, incluso con muchos eventos ocurriendo de manera asíncrona y en distintos lugares, es totalmente posible mantener el orden lógico y la consistencia de toda la transacción. Al final, el sistema funcionó de manera correcta y logramos sincronizar toda la red con éxito.

8. Link de video

https://drive.google.com/file/d/1TVKTP6jQGW7rdOluum835ck_BDwWjk9R/view?usp=sharing

Referencias

- Docker. (2026). *Docker Documentation*. Consultado el 28 de mayo de 2026, desde <https://docs.docker.com/>
- gRPC. (2026). *gRPC Documentation*. Consultado el 28 de mayo de 2026, desde <https://grpc.io/docs/>
- Lamport, L. (1978). Time, Clocks, and the Ordering of Events in a Distributed System. *Communications of the ACM*, 21(7), 558-565. <https://doi.org/10.1145/359545.359563>